Agents

# LLM Agents

Copy page

Language model agents for ARC-AGI-3.

## LLM Agent

- Standard OpenAI API agent that observes the game state and chooses actions using function calling, maintains conversation history with 10-message limit.
- Default Model: gpt-4o-mini
- Usage: `--agent=llm`

## Fast LLM Agent

- Skips the observation step entirely (DO_OBSERVATION=False), making decisions faster but potentially less informed - trades accuracy for speed.
- Default Model: gpt-4o-mini
- Usage: `--agent=fastllm`

## ReasoningLLM

- Uses OpenAI's o4-mini model and captures detailed reasoning metadata including reasoning tokens and thought process in the action.reasoning field.
- Default Model: o4-mini
- Usage: `--agent=reasoningllm`

## GuidedLLM

- Uses the most advanced o3 model with high reasoning effort and includes explicit game-specific rules/strategy in the prompt. This template is for education purposes

### Example Usage

```
# Run LLM agent on a specific game
uv run main.py --agent=llm --game=ls20

# Run fast LLM agent on all games
uv run main.py --agent=fastllm
```

## Benchmarking your LLM agent

If you are comparing prompts, model versions, or agent architectures, use the benchmarking tooling to produce repeatable scorecards and replays. It is designed to get you from zero to benchmarking quickly and works well alongside the LLM agent templates.

- Benchmarking Tooling (BETA)

## Handling Malformed Outputs

LLM agents are expected to return **exactly one** of the valid action names ( `RESET` , `ACTION1` – `ACTION6` ).

In the reference implementation we simply call `.strip()` on the model response and forward the resulting string. In practice a model might return an empty string, additional commentary, or a token that is **not** a valid action. When that happens the agent will raise a `ValueError` and the current game will terminate.

To make your agent more robust you can:

1. **Post-process the model output** – e.g. extract the first word that looks like an action using a regular expression.

```python
import re
import random
from agents.structs import GameAction

def safe_parse(model_response: str) -> GameAction:
    """Return a valid action or raise."""
    # take the FIRST all-caps token that matches a known action
    match = re.search(r"(RESET|ACTION[1-6])", model_response)
    if match:
        action_name = match.group(0).strip()
        try:
            return GameAction.from_name(action_name)
        except ValueError:
            pass
    # fallback - here we pick a random non-RESET action
    valid_actions = [a for a in GameAction if a is not GameAction.RESET]
    return random.choice(valid_actions)
```



‹ Quickstart Benchmarking Agent ›